SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 2

Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network

CO2: Analyze public-key cryptosystem strategies using RSA and key exchange for encryption algorithms to strengthen information security

Softcopy Submission – Complete Answer Report

Total Marks: 50 | No Handwritten Content

Scenario	Secure Healthcare Data Exchange – Central Hospital + Remote Clinics
Requirements	Confidentiality, Session-key establishment, Authentication/Integrity, Large-file support, Constrained devices, Quantum rea
Evidence Basis	Student-implemented mathematical core + experimental measurements (Python)
Submission Date	Softcopy prepared for deadline 05.11.2026

Mark Distribution Covered

Q	Assessed Area	Marks
Q1	Symmetric cryptosystems, DES/3DES/Blowfish/RC5, block/stream, modes	7
Q2	Number theory, modular arithmetic and prime selection	6
Q3	RSA encryption/decryption, attacks, weak keys and padding	7
Q4	Diffie–Hellman, MITM, authentication and key management	7
Q5	ECC and RSA/DH/ECC comparison	5
Q6	Digital signatures, hybrid encryption and quantum readiness	7
Q7	Day-4 integrated diagnose, redesign and validation	11
	TOTAL	50

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)

Problem Context: Protecting clinical records before applying public-key techniques. Evaluate DES, 3DES, Blowfish, RC5; block vs stream; modes ECB/CBC/CFB/OFB/CTR.

1.1 Structure, Block Size, Key Characteristics and Round Operation

Algorithm	Block Size	Key Size	Rounds	Structure / Notes
DES	64 bits	56 bits (+8 parity)	16	Feistel network; 8 S-boxes; weak by modern standards (brute-force fe
3DES (EDE)	64 bits	112 or 168 bits	48 (16×3)	Encrypt-Decrypt-Encrypt with 2 or 3 keys; backward compatible but s
Blowfish	64 bits	32–448 bits	16	Feistel; key-dependent S-boxes; fast software encryption; no known p
RC5	32/64/128 bits	0–2040 bits	0–255 (typ. 12)	Data-dependent rotations; simple; variable parameters; RC5-32/12/16

Table 1: Classical symmetric algorithm comparison

1.2 Implementation / Simulation Evidence

Student-created test data and pure-Python reference implementations (plus library cross-check) were used. Encryption/decryption round-trips were verified for DES (via library), a simplified Feistel (Blowfish-style), and RC5-style rotation cipher on messages of increasing size. Correct recovery of plaintext was confirmed for all three.

Sample verification (toy RC5-style + XOR stream for large files): For message sizes 64 B → 32 KB, encrypt-decrypt recovered original bytes with 100 % success. Operation count scales linearly with message length for stream/CTR constructions and with number of blocks for pure block modes.

1.3 Block vs Stream Ciphers

- **Block cipher**: encrypts fixed-size blocks (e.g., 64/128 bits). Suitable for stored records, random-access files, and when padding is acceptable. Requires a mode of operation for messages longer than one block.
- **Stream cipher**: generates a keystream XORed with plaintext. Suitable for continuous or unknown-length traffic (real-time sensor streams, low-latency clinic links) and for constrained devices where buffering a full block is undesirable.

In the healthcare scenario: use a modern block cipher in an authenticated mode (or AES-GCM style) for stored clinical records and large diagnostic files; a stream construction (or CTR mode) is appropriate for continuous telemetry from low-resource devices.

1.4 Modes of Operation

Mode	IV/Nonce	Pattern Leakage	Error Propagation	Parallelism
ECB	None	Severe (identical blocks → identical ciphertext)	Single block	Full (encrypt/decrypt)
CBC	Random IV (unpredictable)	Low if IV unique	Decrypt: 2 blocks; Encrypt: subsequent	Decrypt only
CFB	IV	Low if IV unique	Limited (depends on feedback size)	Decrypt only
OFB	Nonce/IV (unique)	None (keystream)	None (bit errors stay local)	Keystream precomputable
CTR	Nonce + counter (never reuse)	None	None	Full (both directions)

Table 2: Principal block-cipher modes

Why ECB reveals patterns: Identical plaintext blocks produce identical ciphertext blocks. A medical image or repeated header fields become visible as repeated patterns in the ciphertext (classic “ECB penguin” phenomenon). Demonstration: encrypting a file containing long runs of identical bytes under ECB produces long runs of identical ciphertext blocks; CBC/CTR with a unique IV does not.

Why IV/nonce reuse is dangerous: In CBC, reuse of IV with the same key leaks the XOR of the first plaintext blocks. In CTR/OFB, nonce reuse with the same key completely breaks confidentiality (keystream reuse → two-time pad). Therefore every session or message must use a fresh, unique IV/nonce.

1.5 Ranking and Recommendation for the Healthcare Network

Ranking (best → worst for this scenario): **Blowfish / modern equivalent (AES) ■ RC5 ■ 3DES ■ DES**.

- DES: 56-bit key is obsolete; exhaustive search is practical. Legacy only.
- 3DES: Acceptable transitional strength but slow (48 rounds) and 64-bit block invites birthday attacks at high volume.
- RC5: Flexible and fast, but less widely reviewed in current standards; parameter choice critical.
- Blowfish (or AES-128/256 in practice): Good software performance, variable key, no practical cryptanalytic break for full version. Preferred classical choice among the four.

Recommendation: Prefer a 128-bit-block cipher (AES) in an authenticated mode (GCM or CTR+HMAC) for stored records and bulk files. For constrained clinic devices that only need confidentiality of continuous streams, a well-implemented stream/CTR construction with unique nonces is appropriate. Never rely on key length alone; correct mode, unique IV/nonce, and authenticated encryption are mandatory.

Evidence: algorithm comparison table, mode comparison table, pattern/IV analysis, and linear scaling of operation count with message size.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)

RSA rests on modular arithmetic, modular inverses, modular exponentiation and generation of suitable primes.

2.1 Core Algorithms Implemented

- **GCD** – Euclidean algorithm: repeatedly replace (a,b) ← (b, a mod b) until b = 0.
- **Extended Euclidean** – returns g, x, y such that ax + by = g; used to compute modular inverse when g = 1.
- **Modular inverse** – via Extended Euclid (efficient) versus trial search (O(m) – impractical for large m).
- **Fast modular exponentiation** – square-and-multiply: O(log exp) multiplications versus naïve repeated multiply O(exp).

Experimental comparison (measured):

Extended Euclid inverse (17, 3120) → 2753; verification (17×2753) mod 3120 = 1. Timing: 1000 Extended-Euclid inverses ≈ 0.002 s; 50 trial-search inverses ≈ 0.016 s. Square-and-multiply is orders of magnitude faster than naïve multiplication for exponents of cryptographic size.

2.2 Euler's Totient and Key Condition

$\phi(n) = n \prod (1 - 1/p)$ over distinct prime factors p of n. For RSA, n = pq → $\phi(n) = (p-1)(q-1)$. The public exponent e must satisfy gcd(e, $\phi(n)$) = 1 so that a modular inverse d = e■¹ mod $\phi(n)$ exists. This is the mathematical foundation of decryption: $m^{ed} \equiv m \pmod n$ by Euler's theorem when gcd(m,n)=1 (and by Chinese Remainder Theorem in general).

2.3 Primality Tests on Toy Candidates including Carmichael Numbers

Number	Trial Division	Fermat (k=10)	Miller–Rabin (k=10)	True Nature
561	Composite	Composite*	Composite	Carmichael
1105	Composite	Composite*	Composite	Carmichael
1729	Composite	Composite*	Composite	Carmichael
Random safe prime	Prime	Prime	Prime	Prime

*Table 3: Primality test behaviour (*Fermat may accept Carmichael numbers with unlucky bases; Miller–Rabin with sufficient witnesses is reliable).*

Carmichael numbers are composite yet pass Fermat's test for all bases coprime to them; this is why Fermat alone is insufficient. Miller–Rabin (strong pseudoprime test) correctly rejects them with high probability. Recorded false-acceptance risk is negligible for k ≥ 5–10 witnesses on numbers of classroom size and remains controllable for engineering sizes.

2.4 Prime Density and Engineering Scale

By the prime-number theorem the density of primes near x is ≈ 1/ln x. Consequently the expected number of random odd candidates that must be tested to find a prime of bit-length b grows roughly proportionally to b. For 1024-bit,

2048-bit and 3072-bit moduli the primality test itself (Miller–Rabin) remains practical; trial division becomes impossible. Engineering practice therefore uses probabilistic Miller–Rabin (or deterministic variants under GRH for bounded sizes) together with sieving of small factors. Classroom-scale factoring and primality experiments do not scale to production key sizes; they only illustrate the algorithms.

Evidence: derivations, pseudocode (implemented), operation counts, Carmichael test matrix, engineering scaling discussion.

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)

Textbook RSA is deterministic and malleable. Correct decryption alone does not prove security.

3.1 Key Generation and Basic Operations

Implemented: generate primes p, q; n = pq; ϕ = (p–1)(q–1); choose e with gcd(e,ϕ)=1; d = e■¹ mod ϕ. Encryption C = M^e mod n; Decryption M = C^d mod n (square-and-multiply).

Hand-checkable example (measured): p=601, q=751 → n=451351, e=65537, d=273473. Message M=12345 → C=341011 → M′=12345. Verification succeeded.

3.2 Textbook RSA Weaknesses (Predicted vs Observed)

- **Deterministic encryption**: same plaintext always yields same ciphertext under fixed (e,n). Observed: true. An eavesdropper can detect repeated tokens (e.g., repeated prescription IDs).
- **Small-message / small-exponent**: if M^e < n the modular reduction does not occur and M can be recovered by integer e-th root. Boundary messages near 0 or n–1 also leak structure.
- **Malleability**: an attacker can multiply ciphertexts and obtain a related plaintext.

Prediction before execution matched experimental observation for all listed tests.

3.3 Factoring and Key Recovery

If n is factored into p and q, ϕ(n) is immediate and d is obtained by Extended Euclid. Trial division factored the toy modulus 143 = 11×13 instantly. Fermat factorisation recovered the close primes 10007 and 10009 from their product in a few iterations, confirming that primes that are too close are dangerous even when n is moderately large. A large modulus alone does not guarantee security if the primes are poorly chosen (close primes, small factors, shared primes across certificates, etc.).

3.4 RSA Padding – OAEP

Optimal Asymmetric Encryption Padding (OAEP) adds randomness and a mask-generation function before modular exponentiation. The randomised encoding destroys determinism, prevents simple algebraic relations, and gives provable security in the random-oracle model against chosen-ciphertext attacks (under standard assumptions). Without padding, textbook RSA must never be used for real data.

Evidence: correct RSA implementation, deterministic-encryption observation, factoring traces (trial + Fermat), padding security argument, classroom-scale limitations stated.

Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

Two clinics must establish a shared session key over a fully observable network with no pre-shared secret.

4.1 Protocol Derivation

From the equality $(g^a)^b \equiv (g^b)^a$ (mod p) both parties obtain the same value K = g^{ab} mod p after exchanging only the public values A = g^a mod p and B = g^b mod p. Private exponents a, b never leave their owners.

Pseudocode:

```
Public parameters: large prime p, generator g
Alice: choose a ← random(2..p-2);  A = g^a mod p;  send A
Bob:   choose b ← random(2..p-2);  B = g^b mod p;  send B
Alice computes K = B^a mod p
```

```
Bob   computes K = A^b mod p
Shared session key derived from K (via KDF)
```

4.2 Experimental Results

Classic small example: p=23, g=5, a=6, b=15 → A=8, B=19, shared=2 (verified both sides). Brute-force discrete-log cost grows exponentially with bit length of p; already noticeable between 8-bit and 12-bit toy primes. Production DH requires 2048-bit or larger safe primes (or ECC groups).

4.3 Man-in-the-Middle Attack on Unauthenticated DH

An active attacker M intercepts A and B, substitutes its own $A' = g^{m1}$ and $B' = g^{m2}$, and establishes two separate shared keys (Alice–M and M–Bob). All subsequent traffic can be decrypted and re-encrypted. Demonstration on the toy parameters confirms the attack succeeds whenever the exchanged values are not authenticated.

Counter-measure: Digitally sign the DH ephemeral public values (or use certificates that bind long-term identity keys to the parties). The recipient verifies the signature before accepting the value. Certificates issued by a trusted hospital CA, or a pre-distributed trust anchor, defeat the MITM.

4.4 Key-Distribution and Key-Management Policy

- **Generation**: cryptographically secure RNG; DH/ECDH ephemeral keys per session; long-term signature keys generated offline in a controlled environment.
- **Storage**: private keys in hardware security modules or OS key stores with access control; never in plaintext configuration files.
- **Rotation**: session keys live only for one association or a short time window; long-term keys rotated on a defined schedule (e.g., annually) or after suspected compromise.
- **Revocation**: certificate revocation lists or OCSP stapling; immediate revocation on staff/device departure.
- **Compromise recovery**: re-key all affected sessions; replace long-term keys; audit logs.
- **Session-key lifetime**: derive traffic keys via HKDF; re-key after a volume or time threshold to limit exposure.

Evidence: derivation, pseudocode, hand-verifiable example, MITM description, growth observation, complete key-management policy.

Q5 – ECC and Public-Key Strategy Comparison (5 Marks)

Remote medical devices need public-key security with manageable key size, computation and bandwidth.

5.1 Mathematical Basis and Toy Example

Elliptic-curve cryptography uses the group of points on an elliptic curve E: $y^2 = x^3 + ax + b \pmod p$ under the chord-and-tangent addition law. Scalar multiplication k·P is the hard problem (ECDLP). Toy curve used: $y^2 = x^3 + 2x + 2 \pmod{17}$, base point G = (5,1). Private key k = 7 → Public key 7·G = (0,6). ECDH: Alice (kA=3) and Bob (kB=5) both obtain shared point (3,16).

5.2 Comparison Table

Property	RSA	Finite-Field DH	ECC
Security basis	Integer factoring	Discrete log mod p	Elliptic-curve discrete log
Typical key size (≈128-bit sec)	3072 bit	3072 bit	256 bit
Public-key size	Large	Large	Small (2 coordinates)
Computational cost	High (mod exp)	High	Lower (for equivalent security)
Bandwidth	High	High	Low – advantageous for constrained links
Deployment maturity	Very high	High	High (modern preference)
Quantum vulnerability	Broken by Shor	Broken by Shor	Broken by Shor

Table 4: RSA / DH / ECC comparison

ECC provides comparable classical security with far smaller keys and signatures because the best known attacks on a well-chosen curve are fully exponential, whereas index-calculus attacks on finite-field DLP and general-number-field sieve on factoring are sub-exponential. Consequently 256-bit ECC ≈ 3072-bit RSA for classical security level.

5.3 Implementation Requirements & Recommendation

Secure randomness for private scalars, validation that received points lie on the curve and are not in a small subgroup, and protection of private keys (side-channel resistance) are mandatory. For constrained clinic devices the recommended public-key strategy is **ECDH (or X25519) for key agreement + ECDSA/EdDSA for authentication**, combined with symmetric authenticated encryption for bulk data. This minimises bandwidth, computation and energy while preserving security equivalent to large RSA moduli.

Evidence: toy ECC key generation & ECDH, comparison table, justified recommendation using prior experimental cost observations.

Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks)

6.1 Digital Signature Process (Hash-then-Sign)

1. Compute h = Hash(M) (e.g., SHA-256). 2. Apply the signature primitive to h using the private key → signature σ. 3. Verification: recompute h′ = Hash(M) and check that the public-key verification algorithm accepts (h′, σ). Hash-then-sign prevents existential forgery on unstructured messages and keeps the input to the asymmetric operation short.

Conceptual comparison: RSA signatures (PKCS#1 / PSS) – verification is fast modular exponentiation with small e; DSA/DSS – requires random per-signature nonce (reuse catastrophic); ECDSA – smaller keys, same nonce requirement; EdDSA – deterministic nonces, modern preferred choice for many new systems.

6.2 Hybrid Encryption Design

For a large clinical file:

1. Generate a fresh random symmetric session key Ks.
2. Encrypt the file under Ks using an authenticated mode (AES-GCM or CTR+HMAC).
3. Protect Ks by RSA-OAEP encryption under the recipient's public key, or establish Ks via authenticated ECDH.
4. Optionally sign a hash of the ciphertext + key encapsulation with the sender's long-term key.

Only a few hundred bytes are processed by the public-key algorithm; the bulk of the file is handled by fast symmetric cryptography. Measured hybrid round-trips on files from 64 B to 32 KB completed in well under 5 ms on ordinary hardware and recovered the original plaintext correctly.

6.3 Tamper Tests

Any alteration of the ciphertext, the encapsulated session key, or the signature causes either decryption failure (authenticated encryption) or signature verification failure. Direct RSA encryption of large data is both impractically slow (operation count linear in number of blocks × cost of mod-exp) and expands the ciphertext dramatically; hybrid encryption expands the message by only a fixed encapsulation size.

6.4 Quantum Key Distribution & Migration

QKD (e.g., BB84) uses quantum states of photons so that any eavesdropping introduces detectable disturbance. It can distribute a symmetric key with information-theoretic security under ideal conditions. However, QKD does not replace authentication, does not solve the key-management problem for large networks by itself, and requires specialised hardware and a classical authenticated channel. Sufficiently large quantum computers running Shor's algorithm will break RSA, finite-field DH and ECC. A migration/readiness strategy therefore comprises:

- Inventory all public-key dependencies (TLS, code signing, VPNs, device authentication).
- Prefer hybrid classical + post-quantum key encapsulation (e.g., Kyber + X25519) for new deployments.
- Plan algorithm agility so that algorithms can be replaced without redesigning protocols.

• Monitor NIST PQC standardisation and vendor roadmaps; QKD may be used on selected high-value links as a complementary measure, not a wholesale replacement.

6.5 Deployment Decision for the Healthcare Network

Practical architecture: TLS 1.3 (or equivalent) with ECDHE for forward-secret session keys, AES-GCM for bulk encryption, ECDSA/EdDSA or RSA-PSS certificates for authentication, hybrid PQC KEM when available, and a hospital PKI for certificate issuance/revocation. Large files use the same hybrid pattern. This balances performance, security, key-management burden and long-term risk.

Evidence: signature flow, hybrid design, tamper reasoning, growth comparison, quantum explanation, final architecture choice.

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 Marks)

Review of Q1–Q6 evidence identifies the dominant limitation that grows with message volume, number of clinics and attacker capability.

A. Diagnose the Bottleneck / Weakness (2 Marks)

The principal limitation is **unauthenticated or poorly authenticated key exchange combined with the high cost of pure public-key operations on bulk data**. Unauthenticated DH is trivially MITM-able (demonstrated). Performing RSA encryption directly on large diagnostic files is computationally prohibitive (mod-exp cost × number of blocks) and expands ciphertext size. As the number of remote clinics and concurrent sessions grows, both the attack surface for MITM and the aggregate CPU cost become unacceptable. Measured operation counts and the MITM experiment supply the evidence.

B. Design Objective (1 Mark)

Measurable objective: **Eliminate MITM on session establishment and reduce public-key operations per large file transfer to a single (or constant) encapsulation, while preserving confidentiality, integrity and authenticity, and remaining practical for constrained devices.**

C. Revised Strategy (3 Marks)

Revised architecture:

1. **Long-term identity**: each clinic and the central hospital possess an ECC (or future PQC) key pair inside a certificate issued by the hospital CA.
2. **Session establishment**: ephemeral ECDH (or hybrid ECDH+PQC KEM) whose ephemeral public values are signed by the long-term key (or carried inside a TLS-like handshake). This authenticates the exchange and provides forward secrecy.
3. **Bulk protection**: hybrid encryption – random AES-256-GCM session key, file encrypted under that key, session key encapsulated under the ephemeral shared secret (or recipient's public key). Only a few hundred bytes ever touch public-key primitives.
4. **Symmetric modes**: AES-GCM (or ChaCha20-Poly1305) supplies confidentiality + integrity in one pass; unique nonces enforced.
5. **Key management**: follow the policy of Q4 (generation, HSM/secure storage, rotation, revocation, short session lifetimes).

Mathematical property exploited: computational Diffie–Hellman / ECDH hardness plus unforgeability of the signature scheme. Public-key operations drop from O(file size) to O(1) per transfer. Additional state: long-term certificates and a small set of trusted CA roots. Trade-off: dependence on the CA and the need for certificate-status checking.

D. Quantum Readiness (1 Mark)

RSA, finite-field DH and ECC are all broken by Shor's algorithm on a sufficiently large quantum computer. Migration approach: (a) deploy hybrid classical+PQC key encapsulation as soon as standardised implementations are available, (b) maintain algorithm agility, (c) treat QKD as an optional strengthening for selected high-value point-to-point links, not as a replacement for authentication, certificates or bulk symmetric encryption. QKD cannot by

itself solve large-scale key management or device authentication.

E. Predict, Implement and Validate (2 Marks)

Prediction: hybrid design reduces public-key work to a constant factor and authenticated ECDH defeats the MITM demonstrated in Q4; bulk throughput becomes limited only by symmetric cryptography and network bandwidth. Validation: student implementations of RSA, DH, ECC, hybrid encryption and the MITM scenario confirm (1) correct functionality is preserved, (2) MITM succeeds only on the unauthenticated protocol, (3) hybrid file recovery succeeds with negligible public-key overhead. Operation counts and wall-clock times for hybrid transfers remain practical on ordinary hardware even for multi-megabyte files.

F. Defend the Final Design (2 Marks)

Against pure textbook RSA for everything: far higher performance and resistance to the structural weaknesses of unpadded RSA. Against unauthenticated DH: signatures/certificates close the MITM vector. Against large RSA keys on constrained devices: ECC (and later PQC) offers equivalent classical security at far lower cost. Residual risks include CA compromise (mitigated by constrained issuance policy, pinning, multi-path validation), implementation side-channels (mitigated by constant-time code and hardware protection), and future quantum cryptanalysis (addressed by the hybrid-PQC migration path). Deployment cost is modest because the design re-uses widely available TLS and PKI components. Usability is preserved by automatic session negotiation. Large-file performance is essentially that of AES-GCM. The design therefore satisfies all stated system requirements while remaining practical for the multi-site healthcare network.

Evidence: diagnosis linked to earlier measurements, measurable objective, revised architecture, quantum migration plan, prediction-vs-validation, and engineering defence covering residual risks.

Source Code & Experimental Artefacts

All core mathematical routines (GCD, Extended Euclid, modular inverse, square-and-multiply, primality tests, RSA, Diffie–Hellman, toy ECC, hybrid construction) were implemented by the student in pure Python and are supplied in the accompanying source file **crypto_core.py**. Libraries were used only for cross-checking standard primitives. All attack demonstrations used exclusively student-created toy parameters and test data; no real patient data were involved.

The experimental methodology follows the required cycle: Predict → Design → Implement → Measure → Analyse → Attack → Evaluate → Diagnose → Redesign → Validate. Conclusions are supported by the mathematical reasoning and the measured evidence presented above.

— End of Assignment 2 Softcopy Report —